

MAKE GRADE:

```
make[1]: Leaving directory '/home/rockybhai/xv6-labs-2025'
== Test answers-syscall.txt ==
answers-syscall.txt: OK
== Test sandbox_mask ==
$ make qemu-gdb
sandbox_mask: OK (3.2s)
== Test sandbox_fork ==
$ make qemu-gdb
sandbox_fork: OK (0.4s)
== Test sandbox_path ==
$ make qemu-gdb
sandbox_path: OK (1.3s)
== Test sandbox_most ==
$ make qemu-gdb
sandbox_most: OK (0.8s)
== Test sandbox_minus ==
$ make qemu-gdb
sandbox_minus: OK (0.9s)
== Test attack ==
$ make qemu-gdb
attack: OK (1.0s)
== Test time ==
time: OK
Score: 45/45
rockybhai@DESKTOP-0TFL6FL: /xv6-labs-2025$
```

Using GDB:

- Looking at the backtrace output, which function called syscall?

```
remote Thread 1.1 (src) In: syscall
(gdb) backtrace
#0  syscall () at kernel/syscall.c:133
#1  0x0000000080002602 in usertrap () at kernel/trap.c:68
#2  0x00000003ffffff09c in ?? ()
(gdb)
```

Usertrap() in kernel/trap.c calls syscall

- b. What is the value of `p->trapframe->a7` and what does that value represent? (Hint: look at `user/init.c`, the first user program xv6 starts, and its compiled assembly `user/init.asm`.)

```
(gdb) p /x *p
$1 = {lock = {locked = 0x0, name = 0x80007178, cpu = 0x0}, state = 0x4, chan = 0x0, killed = 0x0, xstate = 0x0,
pid = 0x1, parent = 0x0, kstack = 0x3fffffd000, sz = 0x4000, pagetable = 0x87f52000, trapframe = 0x87f56000,
context = {ra = 0x80001e62, sp = 0x3fffffd0c50, s0 = 0x3fffffd0c80, s1 = 0x800126e8, s2 = 0x800122b8, s3 = 0x0,
s4 = 0x8001dc50, s5 = 0x3, s6 = 0x80023388, s7 = 0x0, s8 = 0x800234b0, s9 = 0x17e, s10 = 0xb0, s11 = 0x2},
ofile = {0x0 <repeats 16 times>}, cwd = 0x800207f8, name = {0x69, 0x6e, 0x69, 0x74, 0x0 <repeats 12 times>}}
```

```
(gdb) p /x p->trapframe->a7
$2 = 0xf
(gdb)
```

0xf

- c. What was the previous mode that the CPU was in?

```
(gdb) p /x $sstatus
$3 = 0x200000022
(gdb)
```

The previous mode was user mode

as it shows `0x200000022` in hex on status which converted to binary then we check bit 8 to check previous mode which is 0 meaning user mode

- d. Write down the assembly instruction the kernel is panicing at. Which register corresponds to the variable `num`?

```
hart 2 starting
scause=0xd sepc=0x80002854 stval=0x0
panic: kerneltrap
qemu-system-riscv64: terminating on signal 15 from pid 36687 ()
rockybh@DESKTOP-9TFLS5L:~/xv6-riscv$ grep 80001bfe kernel/kernel.asm
80001bfe: e8aff0ef      jal    80001288 <uvmmalloc>
```

`80001bfe: e8aff0ef jal 80001288 <uvmmalloc>`, `num = a5`

- e. Why does the kernel crash? Hint: look at figure 3-3 in the text; is address 0 mapped in the kernel address space? Is that confirmed by the value in `scause` above? (See description of `scause` in [RISC-V privileged instructions](#))

The kernel crashes because address 0x0 is not mapped in the kernel's virtual address space (Figure 3-3). The faulted instruction attempts to load from that address. scause confirms this with a load page-fault exception (0xd).

- f. What is the name of the process that was running when the kernel panicked? What is its process id (pid)?

```
(gdb) x/s ((struct proc *)myproc())->name
0x80012830 <proc+344>: "init"
(gdb) p((struct proc *)myproc())->pid
$2 = 1
(gdb)
```

I was getting this error with normal p p->name p p->pid command: value has been optimized out.

Sandbox a command(moderate)

```
hart 2 starting
hart 1 starting
init: starting sh
$ sandbox 32768 - cat README
cat: cannot open README
$ QEMU: Terminated
rockybhair@DESKTOP-9TFLS5L:~/xv6-labs-2025$ ls
LICENSE  Makefile  README  conf  fs.img  grade-lab-syscall  gradelib.py  kernel  mkfs  test-xv6.py  user
rockybhair@DESKTOP-9TFLS5L:~/xv6-labs-2025$ ./grade-lab-syscall sandbox_mask
make: 'kernel/kernel' is up to date.
== Test sandbox_mask == sandbox_mask: OK (1.7s)
```

Sandbox with allowed pathnames(easy)

```
xv6 kernel is booting

hart 2 starting
hart 1 starting
init: starting sh
$ sandbox 32768 README grep xv6 README
xv6 is a re-implementation of Dennis Ritchie's and Ken Thompson's Unix
Version 6 (v6).  xv6 loosely follows the structure and style of v6,
xv6 is inspired by John Lions's Commentary on UNIX 6th Edition (Peer
(kaashoek,rtm@mit.edu).  The main purpose of xv6 is as a teaching
$ sandbox 32768 README grep xv6 x
grep: cannot open x
$
```

Attack xv6 (moderate)

```
hart 2 starting
hart 1 starting
init: starting sh
$ secret xyzzz
$ attack
xyzzz
$
```